

Day 19: Add Payee Using List of Dictionaries

1. Day Number

Day 19

2. Topic Name

List of Dictionaries

3. Connection

Yesterday, you created a **profile dictionary** containing details such as name, phone number, and city.

Today, you will store multiple **payee records**. Each payee will be represented by a dictionary, and all payee dictionaries will be stored inside one list.

```
Profile dictionary
  ↓
One dictionary stores one person's details
  ↓
List of dictionaries
  ↓
Multiple dictionaries store multiple payees
```

4. Important Topics

- List
- Dictionary
- `append()`
- Records

5. Foundational Notes

What is a record?

A **record** is a group of related information about one item or person.

For example, one payee record might contain:

```
{
    "name": "Amit",
    "account_number": "12345678"
}
```

The payee's name and account number belong together, so we store them in one dictionary.

Why use a dictionary?

A dictionary stores information using **keys and values**.

```
"name" → "Amit"  
"account_number" → "12345678"
```

The keys explain what each value represents.

Why use a list?

A bank application may contain several payees.

```
payees = [  
    {"name": "Amit", "account_number": "12345678"},  
    {"name": "Neha", "account_number": "87654321"}  
]
```

Here:

- The outer structure is a **list**.
- Each item inside the list is a **dictionary**.
- Each dictionary represents one payee record.

What does `append()` do?

The `append()` method adds one new item to the end of a list.

```
items.append(new_item)
```

In today's problem, the new item will be a dictionary.

Account numbers should usually be strings

Even when an account number contains only digits, it is generally better to store it as a string.

```
account_number = "00123456"
```

Using a string preserves starting zeros. Converting it to an integer would remove them.

6. Easy Example

This example stores student records:

```
students = []  
  
student = {  
    "name": "Riya",  
    "course": "Python"  
}  
  
students.append(student)  
  
print(students)
```

Output:

```
[{'name': 'Riya', 'course': 'Python'}]
```

Notice the structure:

```
List
└─ Dictionary
    └─ name
    └─ course
```

The same idea can be used for payee records.

7. Problem Statement

Create an empty payee list.

Ask the user to enter:

- Payee name
- Payee account number

Create a dictionary containing the entered details.

Add the payee dictionary to the payee list and display the updated list.

8. Concepts Used

Empty list

The program starts with no saved payees.

```
payee list = empty
```

User input

The program collects the payee name and account number from the user.

Dictionary creation

The entered details are grouped together as one payee record.

```
payee record:
  name → entered payee name
  account number → entered account number
```

List append()

The complete payee record is added to the payee list.

Basic validation

Before saving the payee, the program can check whether:

- The name is empty.
 - A payee with the same name already exists.
-

9. Thought Process

Step 1: Create a container

You need a list that can hold multiple payee records.

Initially, the list should be empty.

Step 2: Ask for payee details

Take the payee name and account number as input.

Clean the name using `strip()` so that spaces-only input is treated as empty.

Step 3: Validate the name

Check whether the cleaned payee name is empty.

If it is empty, do not create the payee record.

Step 4: Check for a duplicate name

Look through the existing payee records.

Compare the new name with each saved payee name. Using lowercase values during comparison can make the duplicate check case-insensitive.

For example:

"Amit" and "amit"

These should usually be treated as the same payee name.

Step 5: Create a dictionary

When the details are valid, create one dictionary containing:

- Payee name
- Account number

Step 6: Add the dictionary to the list

Use `append()` to add the complete dictionary as one list item.

Step 7: Display the result

Print the updated payee list or a success message followed by the saved record.

10. Pseudocode

```
START

Create an empty payee list

Ask the user for payee name
Remove extra spaces from the name

Ask the user for account number
Remove extra spaces from the account number

IF payee name is empty
    Display "Payee name cannot be empty"
ELSE
    Set duplicate found to false

    FOR every saved payee in the payee list
        IF saved payee name matches entered payee name
            Set duplicate found to true

    IF duplicate found is true
        Display "Payee already exists"
    ELSE
        Create a payee dictionary
        Store name in the dictionary
        Store account number in the dictionary

        Add the dictionary to the payee list

        Display success message
        Display the payee list

END
```

11. Suggested Solving Approach: Functional Approach

Organize the main operation inside a function.

The function can receive:

- The current payee list
- The payee name
- The account number

Its responsibility can be:

1. Validate the name.
2. Check whether the name already exists.
3. Create the dictionary.
4. Append the dictionary.
5. Return the updated list.

Possible function design:

```
Function name:
    add_payee

Parameters:
    payee_list
    payee_name
    account_number

Return value:
    updated payee list
```

A function is useful because the same operation can later be called whenever the user chooses **Add Payee** from a banking menu.

12. Easy Edge Cases

Empty payee name

Input:

```
Payee name:
```

After using `strip()`, the name becomes empty.

Expected behaviour:

```
Payee name cannot be empty.
```

The program should not add the record.

Duplicate payee name

Suppose the list already contains:

```
Amit
```

The user enters:

```
amit
```

Expected behaviour:

```
Payee already exists.
```

For a beginner exercise, checking duplicate names is enough. In a real application, duplicate detection may also use account numbers.

13. Expected Input

Example:

```
Enter payee name: Neha Sharma
Enter account number: 0012345678
```

14. Expected Output

```
Payee added successfully.
[{'name': 'Neha Sharma', 'account_number': '0012345678'}]
```

For an empty name:

```
Enter payee name:
Enter account number: 0012345678
Payee name cannot be empty.
```

For a duplicate name:

```
Enter payee name: Neha Sharma
Enter account number: 9988776655
Payee already exists.
```

15. Hint Only

Create the payee information as one dictionary first. Then pass that dictionary to `append()`.

For duplicate checking, loop through the saved dictionaries and access each payee's name using its key:

```
saved_payee["name"]
```

Compare cleaned lowercase names so that "Neha" and "neha" are not saved as separate payees.